

University of Engineering and Technology, New Campus, Lahore
Department of Computer Science

Open Ended Lab

Course Code & Title: CSC303L Compiler Construction

1. Course Learning Outcome:

CLO 4: Develop a compiler by integrating its different phases using automated tools.

Level of CLO: CLO4 is mapped to the 6th level, i.e., Creation of Bloom's taxonomy.

2. Schwab/Herron Levels of Laboratory Openness

Level	Methodology	Description	Problem	Ways & Means	Answers
2	Guided Inquiry	Students investigate a teacher presented question using student designed/selected procedures	Given	Open	Open

3. Problem Statement:

Design and Implementation of a Mini-Compiler for a Subset of C-like Language

Problem Description:

Design and implement a mini-compiler for a restricted subset of a C-like programming language. The compiler should support basic data types (int, float, bool), control structures (if-else, while, for), functions, and various statements such as input, output, assignment, declaration etc.). The compiler must process the source program through lexical analysis, syntax analysis, semantic analysis, and intermediate code generation. Students should integrate error handling and symbol table management into their compilers. The project should simulate how real compilers work and demonstrate system level integration of compiler phases.

Your task is to:

- Implement a **lexical analyzer** (scanner) to tokenize input source code.
- Develop a **syntax analyzer** (parser) to validate structure against grammar rules.
- Create a **semantic analyzer** to handle type checking, variable declarations, and scope resolution.
- Implement a **symbol table** manager and error handler.
- Error Handling and Error Recovery Techniques.

4. Instructions:

- This is a group lab project, and students will work in groups of three members.

- Submit source code, report, and test cases (both valid and invalid).
- Your project will be graded according to the provided rubrics.
- Deadline to submit: 04-01-2026

5. Deliverables:

- Source code of compiler modules (Scanner, parser, semantic analyzer, symbol table).
- Lexical analyzer module that correctly tokenizes the C-like source program.
- Syntax analyzer with defined grammar rules and parse validation for supported constructs.
- Semantic analyzer implementing type checking, scope resolution, and declaration validation.
- Symbol table implementation managing identifiers, data types, and scope information.
- Error handling and error recovery mechanisms with meaningful error messages.
- Test cases demonstrating correct compilation and error detection.
- Report describing design choices, grammar rules, and implementation details.

Open Ended Lab Assessment Rubrics:

Criteria	Excellent(6)	Good(5)	Satisfactory(4)	Needs Improvement(3)	Poor(2-0)
Use of Compiler Concepts	Fully demonstrates all compiler phases with correct integration	Most phases correctly implemented; minor issues in integration	Basic phases present; lacks depth and robustness	Few phases implemented; incomplete or incorrect	No meaningful compiler implementation
System Design & Structure	Well-organized, modular, and extensible compiler architecture	Mostly clear structure; some modularity issues	Functional design but lacks clear separation of phases	Disorganized or inconsistent structure	No logical structure
Functionality	All required features implemented and working perfectly	Most features work with minor limitations	Basic functionality only; limited scope	Incomplete or partially working	Very few or no functional features
Code Quality & Documentation	Clean, well-structured, and fully documented code with report	Good code with some documentation issues	Partially documented; code has readability issues	Minimal documentation; hard to follow	No documentation

Analysis & Comparison	Thorough analysis and insightful comparisons between techniques	Good analysis with some useful comparisons	Basic analysis; limited comparisons	Superficial analysis without depth	No analysis
----------------------------------	---	--	-------------------------------------	------------------------------------	-------------